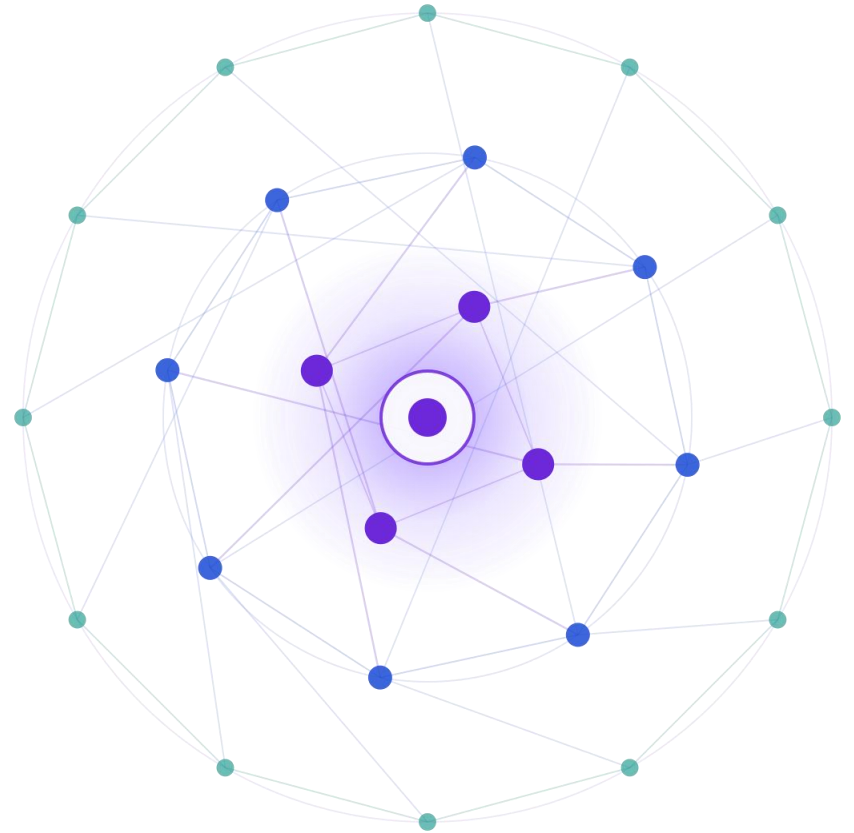


SAMSUNG PRISM

Generative AI Hackathon

3rd Edition 2026 – 27

- Theme ID - 1 (Agentic Code Intelligence)
- Team Name - Error404
- College Name - VIT Vellore (VITV)
- Member Name & Email 1- Chiranjib Dash,
chiranjib.dash2024@vitstudent.ac.in
- Member Name & Email 2- Aditya Ashutosh Mishra
- Member Name & Email 3- Aditya Jaiswal
- Member Name & Email 4- Chirag Goyal
- Submission Github link - <https://github.com/Chiranjib-x/prism-genai-code-retrieval>



Theme

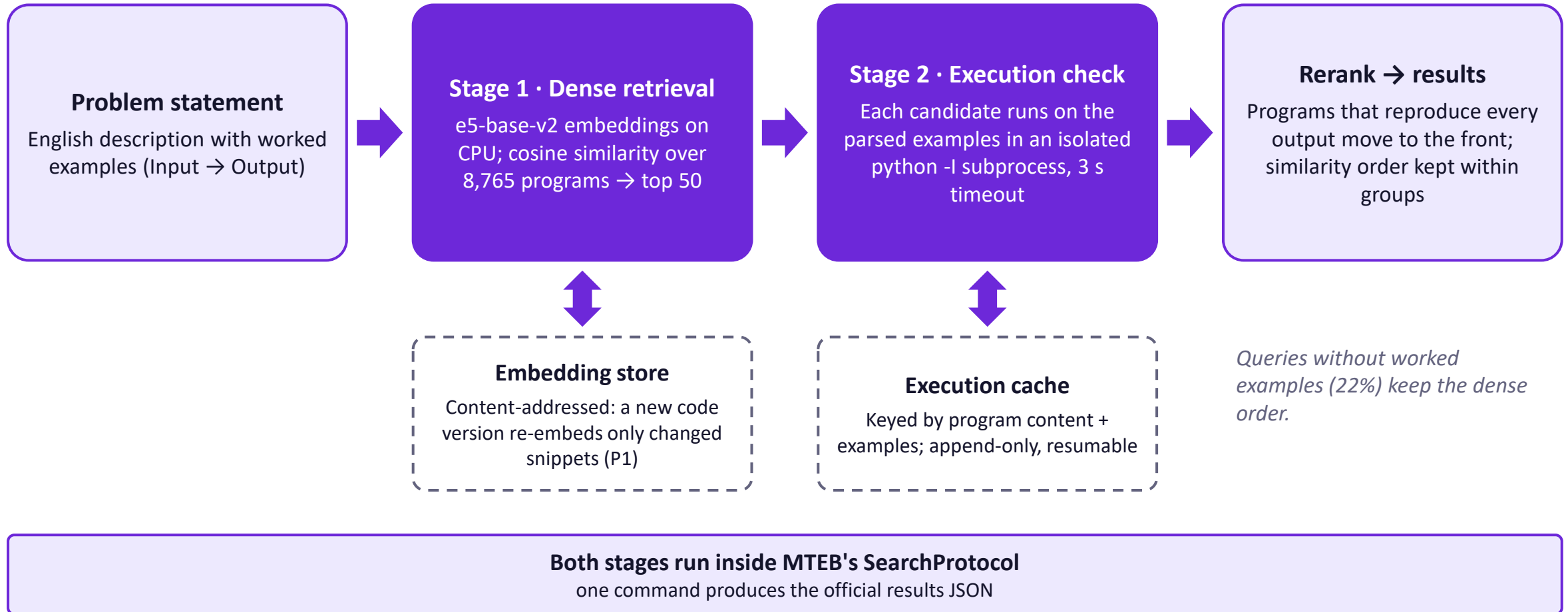
- **Task:** given a programming problem written in plain English, find the Python program that solves it, among 8,765 candidates
- **Benchmark:** ColR AppsRetrieval through MTEB, 3,765 test queries, scored by NDCG@10
- **Constraint:** CPU-only inference, no GPU
- **Scope:** retrieval only; ranking quality is the score, generation is out of scope
- **P1:** retrieval that stays correct as the code corpus changes across versions
- **Our system:** execution-verified code retrieval on CPU. Stage 1 finds candidates by meaning; stage 2 runs them to check they work.

Existing Solutions & Gaps

- **Lexical search:** BM25 scores 0.95 NDCG@10; problem text and code share few words
- **Older code encoders:** UniXcoder 1.36; not trained for English-to-code search
- **General text embedders:** BGE-base 4.05 · E5-base-v2 11.52 · E5-Mistral-7B 21.33 (7B parameters)
- **Code-retrieval embedders:** CodeXEmbed 400M / 2B / 7B: 48.57 / 74.99 / 85.22 · Voyage-Code-002 26.52 (proprietary API)
- **The gap:** every one ranks by similarity. None checks whether a program actually solves the problem, so similar-looking wrong programs outrank correct ones.
- **The unused signal:** 78% of queries contain worked examples (Input → Output), a free and exact test of correctness

All scores are published NDCG@10 on the same benchmark (CoIR and CodeXEmbed papers).

Our Solutions & Architecture Diagram



Demo & Product Walkthrough

- **Run:** `python search.py --qid q5001` (or `--file problem.txt`, or a query string)
- The query has 2 worked examples: 50 candidates executed, 1 passes
- The correct program sits at dense rank 42, well outside the top 10
- Execution moves it to rank 1
- **Speed on CPU:** query encode 261 ms · search 3.4 ms over 8,765 programs · 50 executions 922 ms
- **Demo video:** [demo video link]

```
rank  doc      sim  exec  first line
  1  d5001    0.783 PASS  s = input() <-- gold
  2  d5774    0.802 fail  #~ # MAGIC CODEFORCES PYTHON FAST IO
  3  d6561    0.794 fail  #!/usr/bin/env python3
  4  d7432    0.793 fail  import sys
  5  d1301    0.793 fail  from sys import stdin,stdout
...
gold d5001: dense rank 42 -> final rank 1

timings: model+index load 1.2s | query encode 261 ms |
search 3.4 ms over 8,765 docs | execution 922 ms
```

Actual output of `python search.py --qid q5001` (e5-base-v2, top 50; ranks 6-10 omitted)

Tools and tech stack used

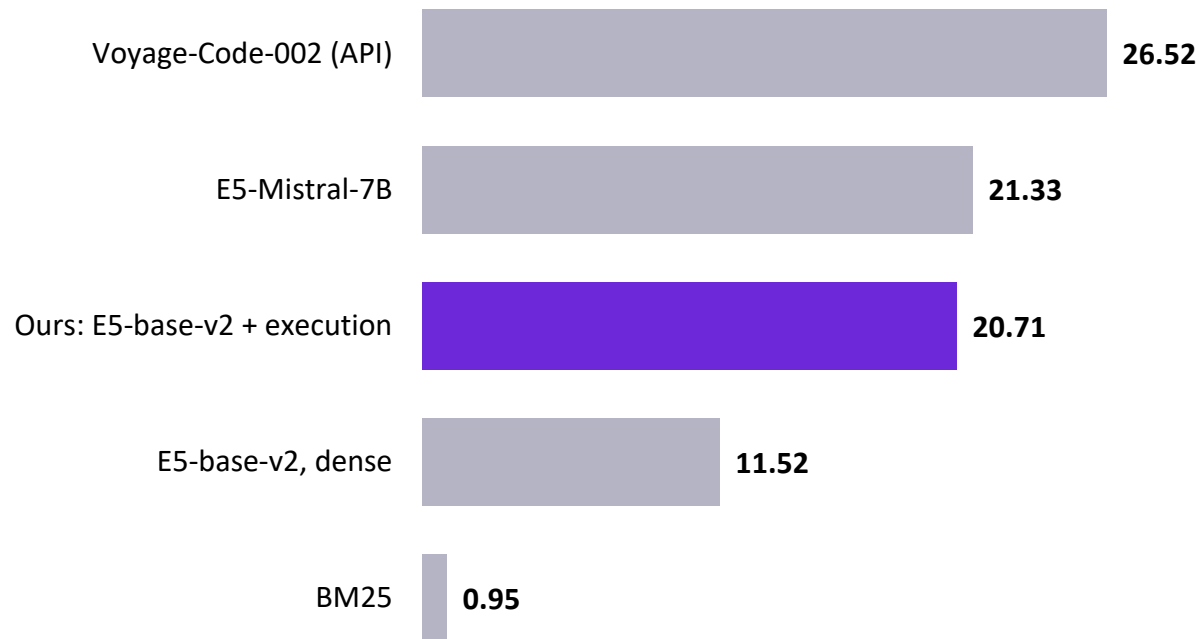
- **Language:** Python 3.11
- **Models:** intfloat/e5-base-v2 (submitted, MIT) · Salesforce/SFR-Embedding-Code-400M_R (integrated, optional, CC-BY-NC-4.0)
- **ML stack:** PyTorch 2.14 (CPU build) · sentence-transformers 6.1 · transformers 5.17 · NumPy
- **Benchmark:** MTEB 2.21.8 · Hugging Face datasets · ColR AppsRetrieval
- **Execution sandbox:** Python subprocess in isolated mode (-I), throwaway temp directory, stripped environment, hard timeout, 24 parallel workers
- **Packaging:** pinned requirements.txt · Dockerfile · MIT licence
- **Development:** built with Claude Code (AI coding assistant); see AI_DISCLOSURE.md

Impact & Use case

- **Code search that returns code that works:** developers and coding agents get candidates that already pass the examples they care about, not just similar-looking code
- **Verification for agentic tools:** an agent can retrieve, execute and keep only candidates that pass, before generating anything new
- **Runs where the code lives:** CPU only. The whole pipeline runs on a laptop or on-premises, and no source code leaves the machine.
- **Evolving codebases (P1):** content-addressed indexes re-embed only snippets that changed, and a stale pass/fail is never served for edited code
- **Cheap to adopt:** the execution stage has no model to train and stacks on top of any embedder

Innovation highlights, results and limitations

NDCG@10, AppsRetrieval test split (3,765 queries)



Other rows: published. Code-retrieval embedders score higher (CodeXEmbed-400M 48.57, published); execution reranking can sit on top of any encoder.

- **Result:** NDCG@10 11.52 → 20.71 (+79.8% relative); MRR@10 9.88 → 19.96 on the full test split
- Level with E5-Mistral-7B using a model ~65× smaller, on CPU
- **Verified:** official MTEB JSON, plus independent rescoring of all 3,765 predictions
- **Limitations:** execution only rescues a correct program found in the top 50 (28% of queries)
 - 22% of queries have no parseable examples and keep the dense order
 - Passing the examples is evidence, not proof of correctness on all inputs
 - Windows sandbox has no memory or network limits; use Docker with --network none
 - CodeXEmbed full run not finished, so no score is claimed for it

What's next

- **Stronger stage 1:** finish CodeXEmbed-400M on CPU (48.57 dense, published, vs 11.52), giving execution a far better pool to rerank
- **Deeper execution:** recall of the correct program rises from 28% at top 50 to 56% at top 500; needs ~1.5M fast, parallel runs
- **Tests where none exist:** for the 22% of queries without examples, generate inputs and rank candidates by agreement (differential testing)
- **Harder sandbox:** containers with --network none and memory caps as the default execution path
- **P1, measured:** re-index an edited corpus version and report what was re-embedded and what changed in the ranking

Brownie points slide(differentiation)

- **Relevance is checked, not estimated:** our largest gain comes from running code, with no extra neural model
- **Honest numbers:** an early 30.74 from a 150-query sample was dropped; only the full-split 20.71 is claimed
- **Reproducible:** one command writes the official MTEB JSON; analyze.py recomputes every score offline in seconds
- **Engineering for current libraries:** Windows DLL import order, MTEB JSON export, CodeXEmbed on transformers 5, fp32 over emulated bf16 on CPU (7.5× faster)
- **Resumable at scale:** 146,800 program executions cached by content; an interrupted run resumes where it stopped
- **Prior art acknowledged:** ExecRetrieval (arXiv 2609.01865) studies execution reranking on APPS. Our contribution is the CPU-only pipeline, scored end to end in MTEB, with versioned indexes.

Checklist- Updated on Public GitHub

- **Working prototype code, public GitHub repo (Y):** <https://github.com/Chiranjib-x/prism-genai-code-retrieval>
- README with reproducible setup instructions (Y)
- **Demo video, max 5 minutes:** [demo video link]
- Presentation file, PPT and PDF (Y)
- **AI disclosure (Y):** AI_DISCLOSURE.md in the repo
- **Release tag (Y):** PRISM_GENAI_HACKATHON_Y2026, with the MTEB results JSON

Thank you

Organised by the Language AI Team and the PRISM Team, Samsung R&D Institute India